

Verification using UVM : Phases

Pravin Zode

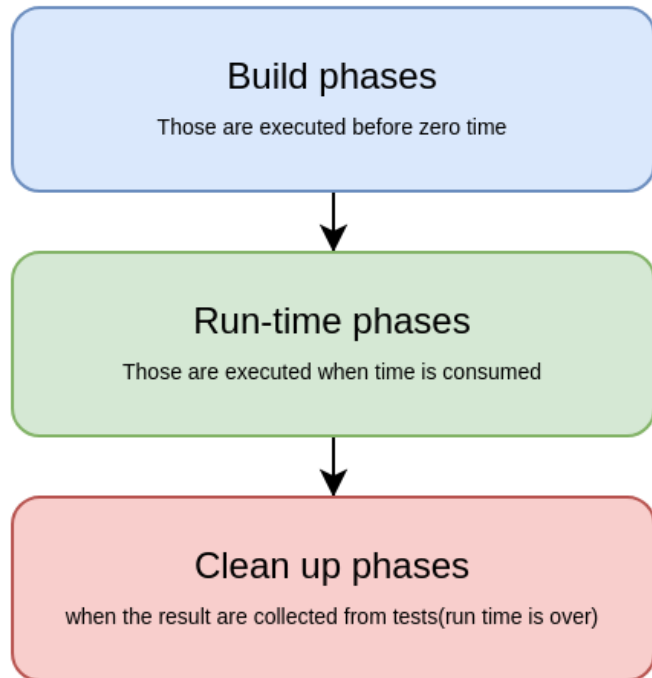
Outline

- UVM Phases
 - Build
 - Run
 - Cleanup
- Objections

Introduction

- Every testbench component derived from `uvm_component` contains predefined phases represented as **callback methods**
- The primary purpose of these phases is to achieve **synchronization**
- A component cannot proceed to the next phase until the current phase execution is completed for **all components in the hierarchy**
- The phasing mechanism is activated by calling **run_test()** from an initial block in the top-level static module
- UVM Phases are executed in a certain order and are virtual methods
- Few phases consume time (tasks) and no time (functions)

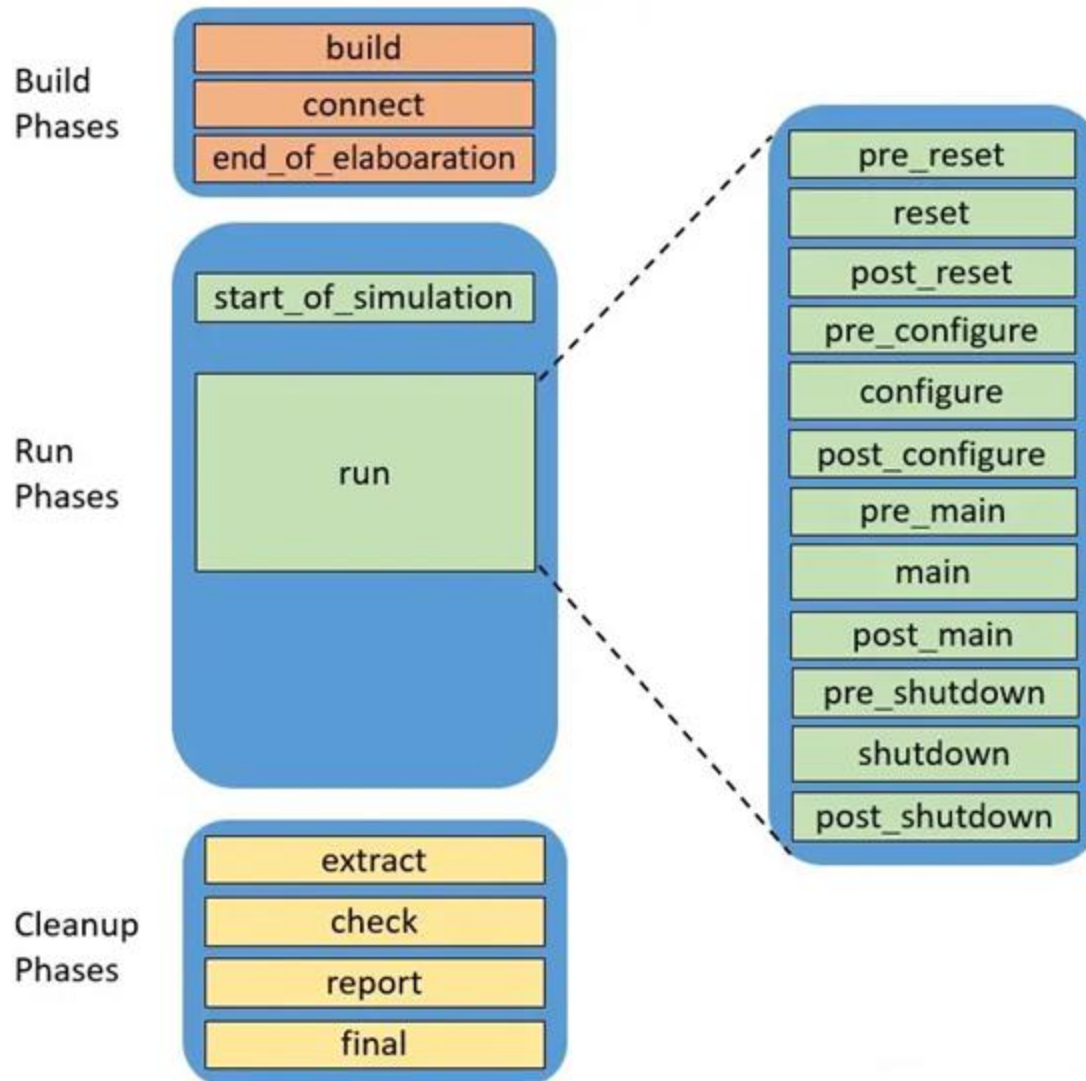
UVM Phases



Main categories in UVM Phases

- **Build phases:** Used to configure or construct the testbench
- **Run-time phases:** Time-consuming testbench activity, like running the test case
- **Clean up phases:** collect and report the results of the simulation

UVM Phases



Build Phase

To construct a testbench, it is necessary to build component objects first and are connected to form a hierarchy

- **Build phases**
- **connect Phase**
- **end_of_elaboration phase**

Build Phase

Phase Type	Phase Name	Description	Execution Approach
Function	Build_phase	Build or create testbench component	Top to bottom
Function	Connect_phase	Connect different testbench component using the TLM port mechanism	Bottom to Top
Function	End_of_elaboration_phase	Before simulation starts, this phase is used to make any final adjustment to the structure, configuration, or connectivity of the testbench, it also displays UVM topology	Bottom to Top

Run Phase

Phase Type	Phase Name	Description
Task	run_phase	✓ Used for stimulus generation ✓ Checks activities of the testbench ✓ Consumes simulation time cycles ✓ The run_phase for all components are executed in parallel

Other phases are also run in parallel to run_phase

Phase Type	Phase Name	Description
Task	pre_reset	✓ Used to add any activity or functionality before reset as power-up signal goes active
Task	Reset	✓ Used to generate a reset and put an interface into its default state
Task	Post_reset	✓ Used to add activity that is required immediately after reset
Task	Pre_configure	✓ After the reset is completed, this phase is used to prepare DUT for configuration programming (update information before it is passed to DUT)

Phase Type	Phase Name	Description
Task	Configure	✓ Used to program the DUT and any memories in the testbench to keep it ready for the start of the test case
Task	Post_configure	✓ Used to wait for the response after configuring DUT or wait for certain DUT state so that main stimulus can be started
Task	Pre_main	✓ Used to ensure that all required components are ready to generate stimulus
Task	Main	✓ Used to apply generated stimulus to the DUT. ✓ Most of the time stimulus are handled using sequences. ✓ This phase completed either all stimuli are exhausted or timeout occurred

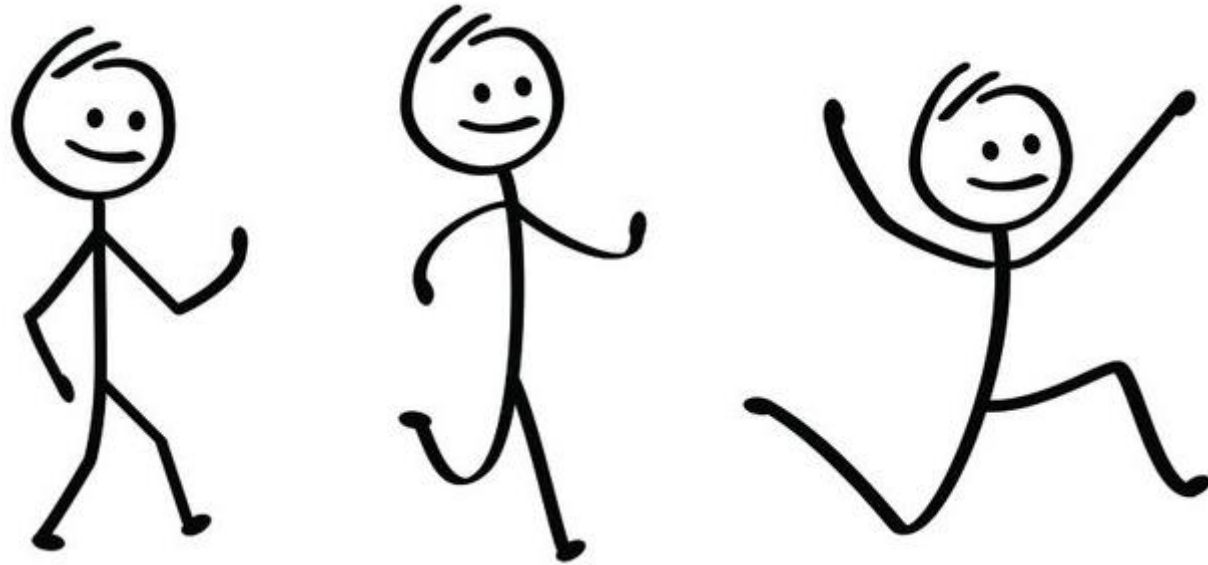
Phase Type	Phase Name	Description
Task	Post_main	✓ Used to take care of any finalization of the main phase
Task	Pre_shutdown	✓ This phase is a buffer for any DUT stimuli that must take care before the shutdown phase
Task	Shutdown	✓ Used to ensure that the effect of stimuli are driven to DUT during main phase that resultant data has drained away. ✓ This phase may also be used to execute any time consuming sequences that read status registers
Task	Post_shutdown	✓ Used to perform any final activity before existing time consuming simulation phases

Cleanup Phase

Phase Type	Phase Name	Description	Execution
Function	Extract	✓ Used to receive and process the information from functional coverage monitors and scoreboards. ✓ This phase also calculate any statistical information that will be used by report_phase	Bottom to Top
Function	Check	✓ Checks DUT behavior and identify for any error that occurred during the execution of the tb.	Bottom to Top
Function	Report	✓ Used to display simulation results. It can also write results to the file	Bottom to Top
Function	final	✓ Used to complete any outstanding actions that are yet to be completed in the testbench	Top to down

Starting UVM Phase Execution

- UVM testbench execution starts using `run_test()`
- Usually called inside initial block, Top-level testbench module
- `run_test()` : Creates UVM root component, starts UVM phasing, and initiates testbench execution
- Root component is usually derived from `uvm_test`
- Root node defines: Test configuration, Stimulus behavior
Verification flow
- `run_test()` can take test name as argument ,
`run_test("my_test");` specifies default test class name



Thank you !

Happy Learning